

TP2

Infrastructure as Code avec Terraform sur Azure

Bloc 4 - Optimisation du SI par l'apport du Cloud Computing

Mastère Dev Manager Full Stack

Cas fil rouge : ShopEasy
Automatiser le déploiement d'une architecture Azure reproductible, versionnée et maîtrisée par le coût.

Format : Travaux pratiques guidés et mise en autonomie
Cloud provider : Microsoft Azure
Outil principal : Terraform
Livrable final : Projet Terraform, preuves d'exécution et note technique

Table des matières

1	Positionnement du TP	3
1.1	Compétences travaillées	3
1.2	Ce qui sera construit	3
2	Rappels courts : Infrastructure as Code et Terraform	3
2.1	Infrastructure as Code	3
2.2	Terraform en quelques notions	4
3	Contexte métier : ShopEasy	4
3.1	Existant fonctionnel	4
3.2	Contraintes	4
3.3	Architecture cible	4
4	Prérequis techniques	5
4.1	Compte et outils	5
4.2	Vérification locale	5
5	Atelier 1 - Initialiser le projet Terraform	6
5.1	Créer l'arborescence	6
5.2	Créer le fichier .gitignore	6
6	Atelier 2 - Déclarer les providers	6
6.1	Fichier versions.tf	6
6.2	Fichier providers.tf	7
7	Atelier 3 - Paramétrer le projet	7
7.1	Variables d'entrée	7
7.2	Fichier terraform.tfvars	8
7.3	Locals et tags	8
8	Atelier 4 - Créer le groupe de ressources et le réseau	8
8.1	Groupe de ressources	8
8.2	Virtual Network et subnet	8
8.3	Questions	9
9	Atelier 5 - Sécuriser le réseau avec un NSG	9
9.1	Créer le Network Security Group	9
9.2	Analyse de sécurité	10
10	Atelier 6 - Déployer deux machines virtuelles Linux	10
10.1	Préparer le script cloud-init	10
10.2	Créer les IP publiques et interfaces réseau	10
10.3	Créer les VM	11
10.4	Questions	12
11	Atelier 7 - Mettre en place un Load Balancer	12
11.1	Créer l'IP publique du Load Balancer	12
11.2	Backend pool, probe et règle	13
11.3	Analyse	13

12 Atelier 8 - Ajouter un Storage Account	13
12.1 Créer un suffixe aléatoire	13
12.2 Créer le Storage Account et un container	14
12.3 Questions FinOps et sécurité	14
13 Atelier 9 - Outputs et validation	14
13.1 Checklist technique	15
14 Atelier 10 - Modifier l'infrastructure	15
14.1 Ajouter un tag	15
14.2 Analyser le plan	15
15 Atelier 11 - Observer une dérive Terraform	16
15.1 Créer une dérive volontaire	16
15.2 Analyse	16
16 Atelier 12 - Préparer un state distant	16
16.1 Structure cible	17
16.2 Questions	17
17 Atelier 13 - Analyse coût, sécurité et maintenabilité	17
17.1 Analyse FinOps	17
17.2 Analyse de sécurité	17
17.3 Maintenabilité	18
18 Atelier 14 - Nettoyage de l'environnement	18
19 Mise en autonomie - Extension du projet	18
19.1 Option A - Subnet privé	18
19.2 Option B - Azure Bastion	18
19.3 Option C - Environnements multiples	19
19.4 Option D - Storage lifecycle	19
20 Livrables attendus	19
21 Quiz de validation	19
21.1 QCM et questions courtes	19
22 Grille d'évaluation	20
23 Corrigé indicatif	21
23.1 Architecture attendue	21
23.2 Réponses attendues aux points clés	21
23.3 Améliorations possibles	21
24 Annexe - Récapitulatif des commandes	21
25 Annexe - Références utiles	22

1 Positionnement du TP

Ce TP fait suite au TP1 Azure consacré au panorama cloud et à la conception d'une architecture applicative. Dans le TP1, les choix d'architecture ont été réalisés principalement sous forme de schémas et d'analyses. Dans ce TP2, l'objectif est de transformer ces choix en infrastructure déclarative avec Terraform.

Idée centrale. Une infrastructure cloud ne doit pas être reconstruite manuellement à chaque environnement. Elle doit pouvoir être décrite dans du code, relue, versionnée, appliquée, détruite et recrée de manière contrôlée.

1.1 Compétences travaillées

- Décrire une infrastructure cloud sous forme de code.
- Automatiser la création de ressources Azure avec Terraform.
- Organiser un projet Terraform maintenable.
- Utiliser les commandes du cycle de vie Terraform : `init`, `fmt`, `validate`, `plan`, `apply`, `destroy`.
- Manipuler variables, outputs, locals et state.
- Déployer une architecture réseau et applicative simple sur Azure.
- Identifier les risques liés au state, aux secrets, aux coûts et aux modifications manuelles.

1.2 Ce qui sera construit

Les apprenants construisent une infrastructure Azure minimale mais réaliste :

- un groupe de ressources ;
- un réseau virtuel Azure ;
- un subnet applicatif ;
- un Network Security Group ;
- deux machines virtuelles Linux configurées automatiquement ;
- une adresse IP publique et un Load Balancer ;
- un Storage Account et un container Blob ;
- des tags, variables, outputs et preuves de validation ;
- une analyse de dérive, de coût et de sécurité.

2 Rappels courts : Infrastructure as Code et Terraform

2.1 Infrastructure as Code

L'Infrastructure as Code, ou IaC, consiste à décrire les ressources d'infrastructure dans des fichiers lisibles par un outil. Ces fichiers deviennent la source de vérité de l'environnement cloud. Ils peuvent être stockés dans Git, relus en revue de code, testés, appliqués et rejoués.

Approche	Mode manuel	Mode IaC
Création	Clics dans le portail Azure	Fichiers Terraform
Reproductibilité	Faible	Forte
Traçabilité	Dépend des journaux	Git + historique Terraform
Risque d'erreur	Élevé	Réduit si les modèles sont relus

Industrialisation	Difficile	Compatible CI/CD
Destruction	Manuelle et risquée	Contrôlée par Terraform

2.2 Terraform en quelques notions

Terraform est un outil déclaratif. L'utilisateur décrit l'état souhaité, puis Terraform calcule les opérations nécessaires pour atteindre cet état.

- **Provider** : plugin permettant de piloter une plateforme, ici Azure via `azurerm`.
- **Resource** : élément créé ou géré, par exemple un VNet ou une VM.
- **Variable** : paramètre d'entrée pour rendre le code réutilisable.
- **Output** : information affichée après déploiement.
- **State** : fichier décrivant les ressources connues de Terraform.
- **Plan** : simulation des changements avant application.

Attention. Le state Terraform peut contenir des informations sensibles. Il ne doit pas être envoyé dans un dépôt public et doit être protégé dans un usage professionnel.

3 Contexte métier : ShopEasy

ShopEasy souhaite industrialiser le déploiement de son environnement Azure. L'équipe a déjà réalisé une première architecture cible lors du TP1. La direction demande maintenant une approche reproductible pour créer un environnement de développement puis préparer les futurs environnements de recette et de production.

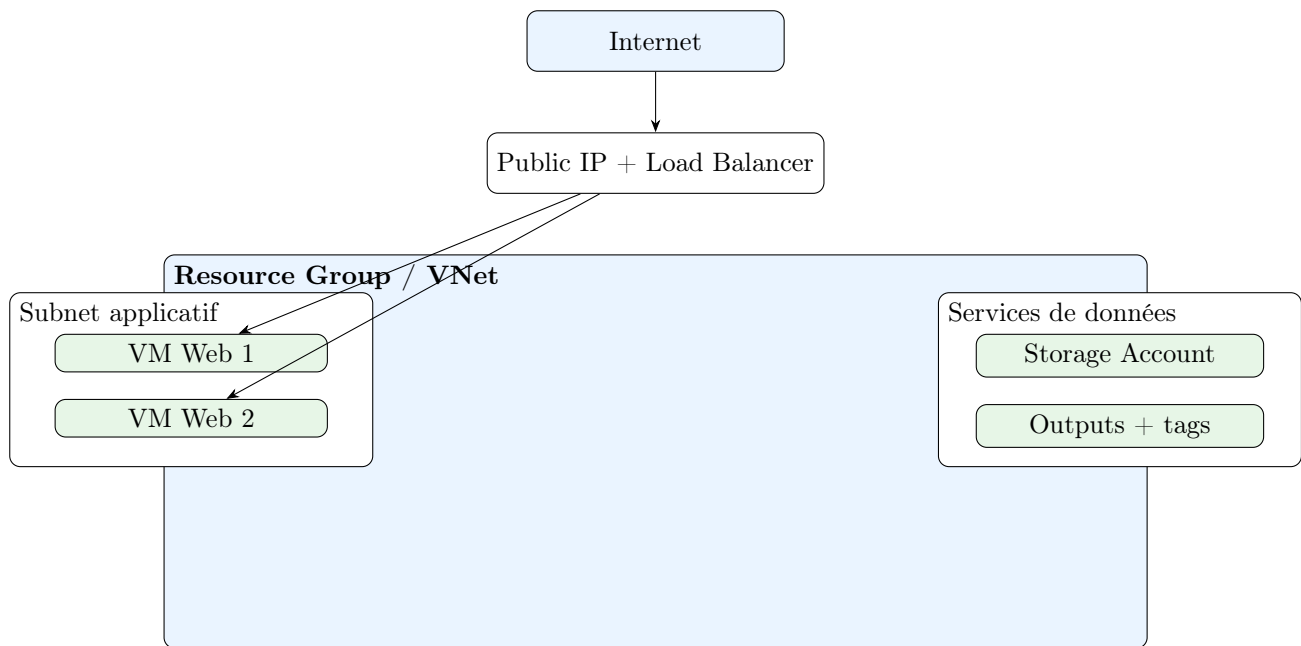
3.1 Existant fonctionnel

L'application ShopEasy est une application web simple. Elle affiche une interface de gestion de commandes et stocke des documents métier. Dans ce TP, la couche applicative sera représentée par des serveurs web Linux minimalistes. L'objectif n'est pas de développer l'application, mais de construire l'infrastructure qui pourrait l'héberger.

3.2 Contraintes

- L'infrastructure doit être créée par Terraform.
- Les ressources doivent être nommées de manière cohérente.
- Les ressources doivent être taguées.
- Les accès réseau doivent être limités.
- Le code doit être organisé en fichiers lisibles.
- La destruction de l'environnement doit être possible en fin de séance.

3.3 Architecture cible



4 Prérequis techniques

4.1 Compte et outils

Chaque apprenant doit disposer :

- d'un accès à une souscription Azure de formation ;
- d'Azure CLI ;
- de Terraform CLI ;
- d'un éditeur de code ;
- d'une clé SSH publique ;
- d'un navigateur pour consulter le portail Azure.

4.2 Vérification locale

Ouvrir un terminal et exécuter :

```
az --version
terraform version
git --version
```

Se connecter à Azure :

```
az login
az account show
```

Si plusieurs souscriptions sont visibles, sélectionner la bonne :

```
az account set --subscription "NOM_OU_ID_DE_LA_SOUSCRIPTION"
```

Sécurité et coût. Le TP crée des ressources cloud réelles. Les ressources doivent être supprimées à la fin avec `terraform destroy`. Les apprenants doivent travailler dans une souscription de formation ou un sandbox encadré.

5 Atelier 1 - Initialiser le projet Terraform

5.1 Créer l'arborescence

À réaliser : créer le dossier de travail suivant :

```
mkdir tp2-terraform-azure
cd tp2-terraform-azure
mkdir templates
```

Créer les fichiers :

```
versions.tf
providers.tf
variables.tf
locals.tf
network.tf
security.tf
compute.tf
loadbalancer.tf
storage.tf
outputs.tf
terraform.tfvars
.gitignore
templates/cloud-init.yml
```

5.2 Créer le fichier .gitignore

```
.terraform/
*.tfstate
*.tfstate.*
*.tfvars.json
crash.log
crash.*.log
.terraform.lock.hcl
```

Point de contrôle : expliquer pourquoi le fichier `terraform.tfstate` ne doit pas être publié dans un dépôt non sécurisé.

6 Atelier 2 - Déclarer les providers

6.1 Fichier versions.tf

Créer le fichier `versions.tf` :

```
terraform {
  required_version = ">= 1.6.0"

  required_providers {
    azurearm = {
      source  = "hashicorp/azurearm"
      version = "~> 4.0"
    }
    random = {
      source  = "hashicorp/random"
      version = "~> 3.6"
    }
  }
}
```

```
}  
}
```

6.2 Fichier providers.tf

```
provider "azurerm" {  
  features {}  
}
```

À réaliser : initialiser Terraform :

```
terraform init  
terraform fmt  
terraform validate
```

Point de contrôle : copier le résultat de `terraform validate` dans le compte rendu.

7 Atelier 3 - Paramétrer le projet

7.1 Variables d'entrée

Créer `variables.tf` :

```
variable "project" {  
  description = "Nom court du projet"  
  type        = string  
  default     = "shopeasy"  
}  
  
variable "environment" {  
  description = "Nom de l'environnement"  
  type        = string  
  default     = "dev"  
}  
  
variable "location" {  
  description = "Region Azure cible"  
  type        = string  
  default     = "francecentral"  
}  
  
variable "admin_username" {  
  description = "Utilisateur administrateur Linux"  
  type        = string  
  default     = "azureuser"  
}  
  
variable "ssh_public_key_path" {  
  description = "Chemin local vers la cle publique SSH"  
  type        = string  
}  
  
variable "allowed_ssh_cidr" {  
  description = "CIDR autorise pour SSH"  
  type        = string  
}
```


7.2 Fichier terraform.tfvars

Compléter terraform.tfvars avec des valeurs adaptées :

```
project      = "shopeasy"
environment  = "dev"
location     = "francecentral"
admin_username = "azureuser"
ssh_public_key_path = "~/.ssh/id_rsa.pub"
allowed_ssh_cidr = "X.X.X.X/32"
```

Ne jamais mettre de mot de passe ou de secret en clair dans un fichier versionné. Pour un contexte professionnel, utiliser une solution de coffre comme Azure Key Vault et une stratégie de variables sécurisées en CI/CD.

7.3 Locals et tags

Créer locals.tf :

```
locals {
  prefix = "${var.project}-${var.environment}"

  common_tags = {
    project      = var.project
    environment  = var.environment
    owner        = "formation"
    managed_by   = "terraform"
  }
}
```

8 Atelier 4 - Créer le groupe de ressources et le réseau

8.1 Groupe de ressources

Dans network.tf, créer le groupe de ressources :

```
resource "azurerm_resource_group" "main" {
  name     = "rg-${local.prefix}"
  location = var.location
  tags     = local.common_tags
}
```

À réaliser : exécuter :

```
terraform fmt
terraform validate
terraform plan
terraform apply
```

Point de contrôle : vérifier dans le portail Azure que le groupe de ressources existe et possède les bons tags.



8.2 Virtual Network et subnet

Compléter network.tf :

```
resource "azurerm_virtual_network" "main" {
  name                = "vnet-${local.prefix}"
  address_space       = ["10.20.0.0/16"]
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  tags                = local.common_tags
}

resource "azurerm_subnet" "web" {
  name                = "snet-web"
  resource_group_name = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.20.1.0/24"]
}
```

À réaliser : appliquer les changements puis vérifier le VNet dans Azure.

8.3 Questions

1. Pourquoi le VNet utilise-t-il une plage privée ?
2. Que faudrait-il ajouter pour isoler une base de données dans un subnet dédié ?
3. Pourquoi séparer les fichiers Terraform au lieu de tout placer dans `main.tf` ?

9 Atelier 5 - Sécuriser le réseau avec un NSG

9.1 Créer le Network Security Group

Dans `security.tf` :

```
resource "azurerm_network_security_group" "web" {
  name                = "nsg-${local.prefix}-web"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  tags                = local.common_tags

  security_rule {
    name                = "Allow-HTTP"
    priority            = 100
    direction          = "Inbound"
    access              = "Allow"
    protocol            = "Tcp"
    source_port_range   = "*"
    destination_port_range = "80"
    source_address_prefix = "Internet"
    destination_address_prefix = "*"
  }

  security_rule {
    name                = "Allow-SSH-Admin"
    priority            = 110
    direction          = "Inbound"
    access              = "Allow"
  }
}
```

```

    protocol          = "Tcp"
    source_port_range  = "*"
    destination_port_range = "22"
    source_address_prefix = var.allowed_ssh_cidr
    destination_address_prefix = "*"
  }
}

resource "azurerm_subnet_network_security_group_association" "web" {
  subnet_id           = azurerm_subnet.web.id
  network_security_group_id = azurerm_network_security_group.web.id
}

```

À réaliser : appliquer la configuration et vérifier l'association subnet/NSG dans Azure.

Question de recul. Dans un environnement de production, l'accès SSH direct depuis Internet serait évité ou très fortement encadré. On privilégierait Azure Bastion, un VPN, un jump server contrôlé ou une approche d'administration sans accès direct.

9.2 Analyse de sécurité

Compléter le tableau.

Flux	Autorisé ?	Justification	Risque résiduel
Internet vers HTTP			
SSH depuis votre IP			
SSH depuis Internet complet			
Tout trafic sortant			

10 Atelier 6 - Déployer deux machines virtuelles Linux

10.1 Préparer le script cloud-init

Créer templates/cloud-init.yml :

```

#cloud-config
package_update: true
packages:
  - nginx
write_files:
  - path: /var/www/html/index.html
    content: |
      <html>
        <body>
          <h1>ShopEasy - serveur web ${server_index}</h1>
          <p>Deploiement realise avec Terraform sur Azure.</p>
        </body>
      </html>
runcmd:
  - systemctl enable nginx
  - systemctl restart nginx

```

10.2 Créer les IP publiques et interfaces réseau

Dans compute.tf :

```

resource "azurerm_public_ip" "web" {
  count          = 2
  name           = "pip-${local.prefix}-web-${count.index + 1}"
  location       = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  allocation_method = "Static"
  sku            = "Standard"
  tags           = local.common_tags
}

resource "azurerm_network_interface" "web" {
  count          = 2
  name           = "nic-${local.prefix}-web-${count.index + 1}"
  location       = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  tags           = local.common_tags

  ip_configuration {
    name                       = "internal"
    subnet_id                 = azurerm_subnet.web.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id       = azurerm_public_ip.web[count.index].id
  }
}

```

10.3 Créer les VM

Toujours dans `compute.tf` :

```

resource "azurerm_linux_virtual_machine" "web" {
  count          = 2
  name           = "vm-${local.prefix}-web-${count.index + 1}"
  location       = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  size           = "Standard_B1s"
  admin_username = var.admin_username
  network_interface_ids = [
    azurerm_network_interface.web[count.index].id
  ]

  admin_ssh_key {
    username   = var.admin_username
    public_key = file(var.ssh_public_key_path)
  }

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }

  custom_data = base64encode(templatefile("${path.module}/templates/cloud-init.yml", {
    server_index = count.index + 1
  }))

  tags = local.common_tags
}

```

À réaliser : exécuter :

```
terraform fmt
terraform validate
terraform plan
terraform apply
```

Point de contrôle : récupérer les IP publiques des VM et tester dans un navigateur.

```
terraform state list
az vm list -g rg-shopeasy-dev -o table
```

10.4 Questions

1. À quoi sert `count` dans cette configuration ?
2. Quel est le rôle de `custom_data` ?
3. Pourquoi le type `Standard_B1s` est-il acceptable pour un environnement de formation ?
4. Pourquoi ne faut-il pas conserver des IP publiques directes sur les VM en production ?

11 Atelier 7 - Mettre en place un Load Balancer

11.1 Créer l'IP publique du Load Balancer

Dans `loadbalancer.tf` :

```
resource "azurerm_public_ip" "lb" {
  name                = "pip-${local.prefix}-lb"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  allocation_method   = "Static"
  sku                 = "Standard"
  tags                = local.common_tags
}

resource "azurerm_lb" "web" {
  name                = "lb-${local.prefix}-web"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  sku                 = "Standard"
  tags                = local.common_tags

  frontend_ip_configuration {
    name                = "public-frontend"
    public_ip_address_id = azurerm_public_ip.lb.id
  }
}
```

11.2 Backend pool, probe et règle

```
resource "azurerm_lb_backend_address_pool" "web" {
  name          = "backend-web"
  loadbalancer_id = azurerm_lb.web.id
}

resource "azurerm_network_interface_backend_address_pool_association" "web" {
  count          = 2
  network_interface_id = azurerm_network_interface.web[count.index].id
  ip_configuration_name = "internal"
  backend_address_pool_id = azurerm_lb_backend_address_pool.web.id
}

resource "azurerm_lb_probe" "http" {
  name          = "http-probe"
  loadbalancer_id = azurerm_lb.web.id
  protocol      = "Http"
  request_path   = "/"
  port          = 80
}

resource "azurerm_lb_rule" "http" {
  name          = "http-rule"
  loadbalancer_id = azurerm_lb.web.id
  protocol      = "Tcp"
  frontend_port = 80
  backend_port  = 80
  frontend_ip_configuration_name = "public-frontend"
  backend_address_pool_ids = [azurerm_lb_backend_address_pool.web.id]
  probe_id       = azurerm_lb_probe.http.id
}
```

À réaliser : appliquer puis tester l'adresse IP du Load Balancer dans un navigateur.

Point de contrôle : rafraîchir plusieurs fois la page et observer si les réponses alternent entre les deux serveurs.

11.3 Analyse

1. Quel problème résout le Load Balancer ?
2. Que se passe-t-il si une VM devient indisponible ?
3. Quelle différence y aurait-il avec Azure Application Gateway ?

12 Atelier 8 - Ajouter un Storage Account

12.1 Créer un suffixe aléatoire

Les noms de Storage Account doivent être globalement uniques. Ajouter dans `storage.tf` :

```
resource "random_string" "suffix" {
  length = 6
  upper  = false
}
```

```
special = false
}
```

12.2 Créer le Storage Account et un container

```
resource "azurerm_storage_account" "docs" {
  name                        = "${replace(local.prefix, "-", "")}docs${random_string.suffix.result}"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location
  account_tier                = "Standard"
  account_replication_type    = "LRS"
  min_tls_version             = "TLS1_2"
  tags                       = local.common_tags

  blob_properties {
    versioning_enabled = true
  }
}

resource "azurerm_storage_container" "documents" {
  name            = "documents"
  storage_account_id = azurerm_storage_account.docs.id
  container_access_type = "private"
}
```

À réaliser : appliquer puis vérifier :

- le Storage Account ;
- le container privé ;
- l'activation du versioning ;
- les tags.

12.3 Questions FinOps et sécurité

1. Pourquoi le container doit-il être privé ?
2. Pourquoi le versioning peut-il augmenter les coûts ?
3. Quelle règle de cycle de vie proposer pour limiter les coûts ?

13 Atelier 9 - Outputs et validation

Créer `outputs.tf` :

```
output "resource_group_name" {
  value = azurerm_resource_group.main.name
}

output "load_balancer_public_ip" {
  value = azurerm_public_ip.lb.ip_address
}
```

```
output "web_vm_public_ips" {
  value = azurerm_public_ip.web[*].ip_address
}

output "storage_account_name" {
  value = azurerm_storage_account.docs.name
}
```

À réaliser : exécuter :

```
terraform output
terraform show
```

Livrable attendu : une capture des outputs et une capture de la page web accessible via le Load Balancer.

13.1 Checklist technique

Contrôle	OK / KO	Preuve
Le projet Terraform s'initialise sans erreur		
<code>terraform validate</code> réussit		
Le Resource Group est créé		
Le VNet et le subnet existent		
Le NSG limite SSH à votre IP		
Deux VM Linux sont créées		
Nginx répond sur les VM		
Le Load Balancer répond en HTTP		
Le Storage Account est privé		
Le versioning Blob est activé		
Les ressources sont taguées		

14 Atelier 10 - Modifier l'infrastructure

14.1 Ajouter un tag

Modifier `locals.tf` pour ajouter un tag :

```
common_tags = {
  project      = var.project
  environment  = var.environment
  owner        = "formation"
  managed_by   = "terraform"
  cost_center  = "cloud-training"
}
```

À réaliser : lancer :

```
terraform plan
terraform apply
```

14.2 Analyser le plan

Avant d'appliquer, répondre :

1. Terraform prévoit-il de recréer toutes les ressources ?
2. Quelles ressources sont simplement mises à jour ?
3. Pourquoi le plan est-il indispensable avant application ?

15 Atelier 11 - Observer une dérive Terraform

15.1 Créer une dérive volontaire

Dans le portail Azure, modifier manuellement un tag sur le Resource Group, par exemple :

```
manual_change = true
```

Puis revenir dans le terminal et exécuter :

```
terraform plan
```

15.2 Analyse

1. Terraform détecte-t-il une différence ?
2. Quelle action propose-t-il ?
3. Pourquoi les modifications manuelles sont-elles dangereuses dans une organisation ?
4. Quelle règle d'équipe proposer pour limiter ce risque ?

Bonne pratique. Dans une organisation mature, les changements d'infrastructure passent par une modification du code Terraform, une revue de code, un plan, puis une application contrôlée. Le portail Azure sert surtout à l'observation et au diagnostic.

16 Atelier 12 - Préparer un state distant

Cet atelier est une préparation conceptuelle. En production, le state local est insuffisant pour un travail d'équipe. Il faut un backend distant sécurisé, souvent basé sur un Storage Account Azure avec verrouillage et contrôle d'accès.

16.1 Structure cible

Un backend distant Azure peut prendre cette forme :

```
terraform {
  backend "azurerm" {
    resource_group_name = "rg-terraform-state"
    storage_account_name = "stterraformstatexxx"
    container_name      = "tfstate"
    key                 = "shopeasy/dev/terraform.tfstate"
  }
}
```

Ne pas ajouter ce bloc sans avoir créé au préalable le Resource Group, le Storage Account et le container de state. Dans un vrai projet, cette partie est initialisée par une équipe plateforme ou par un bootstrap dédié.

16.2 Questions

1. Pourquoi le state distant facilite-t-il le travail en équipe ?
2. Pourquoi faut-il protéger l'accès au Storage Account du state ?
3. Pourquoi faut-il séparer le state de développement, recette et production ?

17 Atelier 13 - Analyse coût, sécurité et maintenabilité

17.1 Analyse FinOps

Compléter le tableau suivant.

Ressource	Coût relatif	Risque de surcoût	Optimisation proposée
VM Linux			
IP publiques			
Load Balancer			
Storage Account			
Versioning Blob			
Disques managés			

17.2 Analyse de sécurité

Risque	Cause possible	Impact	Correction
--------	----------------	--------	------------

SSH trop ouvert
State exposé
Storage public
Tags absents
Secrets dans Git
Modification manuelle

17.3 Maintenabilité

Répondre aux questions :

1. Comment rendre ce projet réutilisable pour un environnement de recette ?
2. Quelles variables faudrait-il ajouter ?
3. Quels fichiers pourraient devenir des modules Terraform ?
4. Quelles validations automatiques pourrait-on ajouter dans une pipeline CI/CD ?

18 Atelier 14 - Nettoyage de l'environnement

À la fin du TP, supprimer les ressources créées :

```
terraform plan -destroy  
terraform destroy
```

Vérifier ensuite dans Azure que le groupe de ressources a disparu ou qu'il ne contient plus de ressources facturables.

Point obligatoire. Ne jamais quitter la séance sans avoir supprimé les ressources de formation, sauf consigne explicite du formateur.

19 Mise en autonomie - Extension du projet

Les apprenants doivent proposer une amélioration de l'infrastructure Terraform. Choisir une des options suivantes.

19.1 Option A - Subnet privé

Ajouter un subnet privé destiné à accueillir une future base de données. Le subnet ne doit pas exposer de ressource publique.

19.2 Option B - Azure Bastion

Proposer une évolution où les VM n'ont plus d'IP publique directe. L'administration passe par Azure Bastion ou par un accès privé contrôlé.

19.3 Option C - Environnements multiples

Adapter les variables pour pouvoir déployer `dev`, `test` et `prod` sans dupliquer tout le code.

19.4 Option D - Storage lifecycle

Proposer une stratégie de cycle de vie pour limiter les coûts liés au versioning Blob.

Livrable attendu : un court document décrivant la modification choisie, les fichiers impactés et les risques associés.

20 Livrables attendus

À la fin du TP, chaque groupe remet :

1. l'arborescence complète du projet Terraform ;
2. les fichiers `.tf` ;
3. les captures des commandes `init`, `validate`, `plan`, `apply`, `output` ;
4. une capture du portail Azure montrant les ressources ;
5. une capture de la page web via le Load Balancer ;
6. l'analyse de dérive ;
7. les tableaux FinOps et sécurité complétés ;
8. une note technique courte présentant les choix réalisés.

21 Quiz de validation

21.1 QCM et questions courtes

1. Terraform est-il un outil impératif ou déclaratif ? Justifier.
2. Quel est le rôle d'un provider Terraform ?
3. À quoi sert le fichier `terraform.tfstate` ?
4. Pourquoi exécuter `terraform plan` avant `terraform apply` ?
5. Quelle commande formate le code Terraform ?
6. Quelle commande vérifie la syntaxe et la cohérence de base ?
7. Quel service Azure correspond au réseau privé logique ?
8. Quel composant Azure filtre les flux réseau entrants et sortants ?
9. Pourquoi restreindre SSH à une seule adresse IP ?
10. Quel est l'intérêt de `count` dans le déploiement des VM ?
11. Pourquoi utiliser des variables ?
12. Pourquoi utiliser des outputs ?
13. Pourquoi taguer les ressources ?
14. Quelle est la différence entre un changement Terraform et un changement manuel dans le portail ?
15. Quel risque présente un Storage Account public ?
16. Pourquoi le versioning Blob peut-il générer des coûts supplémentaires ?
17. À quoi sert un Load Balancer ?
18. Pourquoi un state distant est-il préférable en équipe ?

19. Citer deux bonnes pratiques de sécurité pour un projet Terraform.
20. Citer deux améliorations possibles pour rendre l'architecture plus proche d'un environnement de production.

22 Grille d'évaluation

Critère	Points	Indicateurs
Structure du projet Terraform	3	fichiers lisibles, nommage clair, séparation logique
Provider, variables et tags	3	configuration propre, variables utiles, tags cohérents
Réseau Azure	4	Resource Group, VNet, subnet, NSG fonctionnels
Compute	4	deux VM déployées, cloud-init, accès web validé
Load Balancer	3	IP publique, backend pool, probe, règle HTTP
Storage	2	Storage Account privé, container, versioning
Utilisation du workflow Terraform	4	init, fmt, validate, plan, apply, outputs, destroy
Analyse sécurité et FinOps	4	risques identifiés, mesures pertinentes, coûts maîtrisés
Analyse de dérive	2	drift observé et expliqué
Qualité des livrables	3	captures, explications, note technique
Total	32	

23 Corrigé indicatif

23.1 Architecture attendue

Une solution correcte contient au minimum :

- un Resource Group dédié ;
- un VNet avec un subnet applicatif ;
- un NSG autorisant HTTP depuis Internet et SSH uniquement depuis l'adresse de l'apprenant ;
- deux VM Linux créées avec `count` ;
- un script `cloud-init` installant Nginx ;
- un Load Balancer public dirigeant le trafic vers les VM ;
- un Storage Account privé avec versioning Blob ;
- des outputs utiles ;
- des tags de projet, environnement, propriétaire et gestionnaire.

23.2 Réponses attendues aux points clés

Question	Réponse attendue
Pourquoi Terraform ?	Pour rendre l'infrastructure reproductible, versionnée, contrôlée et automatisable.
Pourquoi <code>plan</code> ?	Pour prévisualiser les changements avant d'impacter Azure.
Pourquoi protéger le state ?	Il contient la représentation des ressources et peut contenir des données sensibles.
Pourquoi des variables ?	Pour éviter le code en dur et réutiliser le projet sur plusieurs environnements.
Pourquoi des tags ?	Pour identifier les coûts, le propriétaire, l'environnement et faciliter la gouvernance.
Pourquoi limiter SSH ?	Pour réduire la surface d'attaque.
Pourquoi un Load Balancer ?	Pour répartir le trafic et améliorer la disponibilité.
Pourquoi éviter les modifications manuelles ?	Elles créent une dérive entre le code et l'infrastructure réelle.

23.3 Améliorations possibles

- Supprimer les IP publiques des VM et utiliser Azure Bastion.
- Ajouter un subnet privé pour les données.
- Externaliser le state Terraform dans un backend Azure sécurisé.
- Ajouter une pipeline CI/CD avec `terraform fmt`, `validate`, `plan` et approbation.
- Remplacer les VM par Azure App Service si l'objectif est de réduire l'administration système.
- Ajouter Azure Monitor et Log Analytics pour l'observabilité.
- Ajouter des règles de cycle de vie sur le Storage Account.

24 Annexe - Récapitulatif des commandes

```
az login
az account show
az account set --subscription "NOM_OU_ID"

terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
terraform output
terraform state list
terraform plan -destroy
terraform destroy
```

25 Annexe - Références utiles

- Documentation Terraform - Azure Provider : <https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs>
- Tutoriels Terraform Azure : <https://developer.hashicorp.com/terraform/tutorials/azure-get-started>
- Microsoft Learn - Terraform sur Azure : <https://learn.microsoft.com/azure/developer/terraform/>
- Microsoft Learn - Virtual Networks : <https://learn.microsoft.com/azure/virtual-network/>
- Microsoft Learn - Network Security Groups : <https://learn.microsoft.com/azure/virtual-network/network-security-groups-overview>
- Microsoft Learn - Azure Pricing Calculator : <https://learn.microsoft.com/azure/cost-management-billing/costs/pricing-calculator>